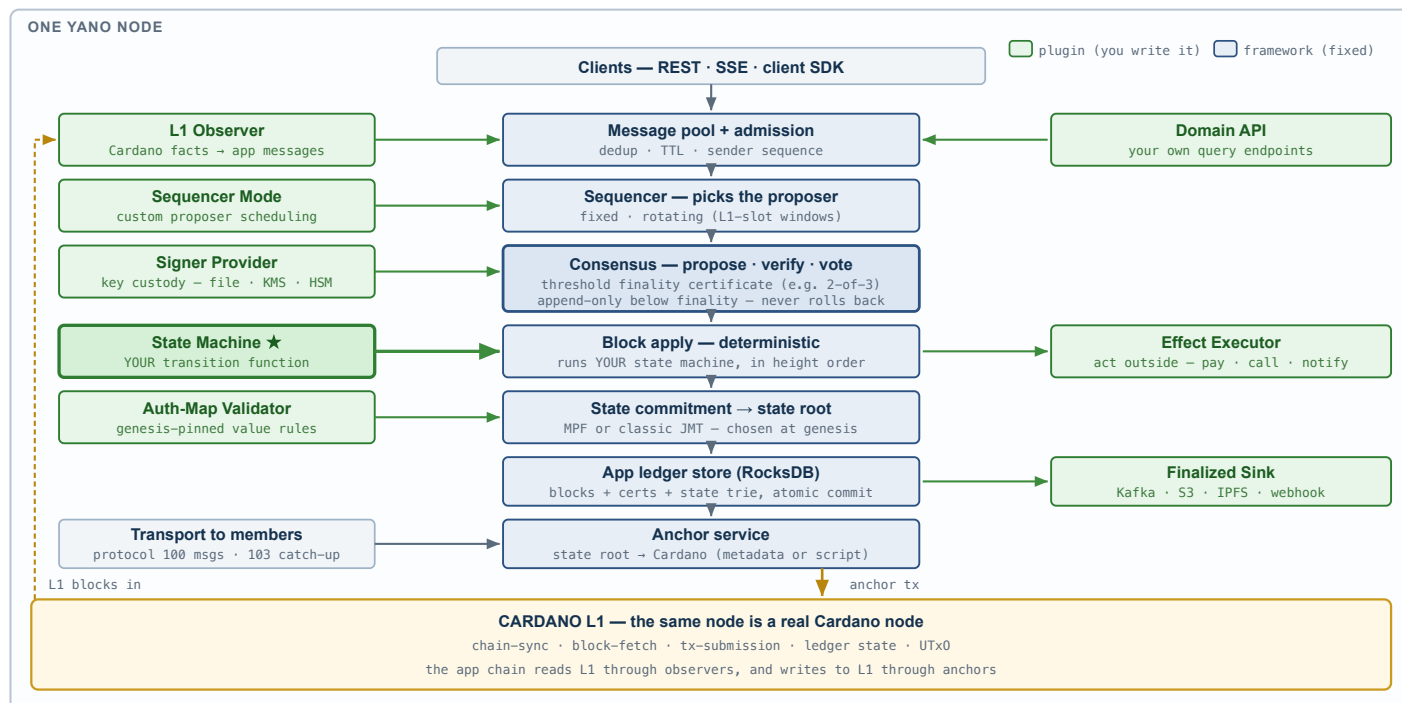


Yano App Chain — what it is, and what its parts are

A **Yano app chain** is a small, permissioned ledger that runs *inside* a Cardano node, beside the Cardano chain. A fixed group of members orders messages, applies them through a deterministic state machine, and signs the result. Every few blocks the resulting state root is **anchored onto Cardano L1**, so anyone can later prove what the group agreed — using nothing but a Cardano transaction. The framework supplies networking, ordering, finality, storage, proofs and anchoring; **you supply the transition function.**

The components — green boxes are yours (plugins), blue boxes are the framework



The one component you almost always write

The **State Machine** ★. Everything else has a working default. The framework never looks inside a message body — your machine is the only layer that interprets it. A chain with an off-the-shelf machine (`ordered-log`, `kv-registry`, `eutxo-ledger`, `authenticated-map`) needs **no code at all**, only configuration.

What the framework will never let you change

Consensus and the finality certificate, block ordering, the wire protocols, the ledger store, catch-up, and membership rules. These are fixed so that every member computes the same chain. Extension points exist only where a choice cannot break agreement — or where it is verified by every member.

How a message becomes proven state

Every member runs the same pipeline. Nothing is trusted: a follower re-derives everything the proposer claims — the messages, the state root, the L1 facts — before it will sign.



Your state machine runs at step 3 AND step 5 — on every member, never only on the proposer.
That is why it must be deterministic: if two members compute different state, the block cannot reach threshold and the chain stops rather than forks.

A member that is behind (restart, new joiner) fetches finalized blocks over **protocol 103**, replays them through the same verification, and rejoins. Nothing is taken on faith.

The nine plugin extension points

Extension point	SPI (ServiceLoader)	What it lets you do · shipped example
App state machine ★	AppStateMachineProvider	Interpret message bodies and own the state. <i>ordered-log, kv-registry, eutxo-ledger, authenticated-map, composite</i>
L1 observer	L1observerProvider	Turn a Cardano fact into an app message. <i>address-deposit, metadata-label, eutxo-vault-deposit</i>
Effect executor	AppEffectExecutorFactory	Act on the outside world after finality. <i>Cardano payment</i>
Finalized sink	FinalizedStreamSinkFactory	Stream finalized blocks out. <i>Kafka, S3, IPFS, webhook</i>
Sequencer mode	SequencerModeProvider	Decide who proposes. <i>fixed, rotating (L1-slot windows)</i>
Signer provider	SignerProviderFactory	Where the member key lives — file, KMS, HSM
Auth-map validator	AuthenticatedMapValueValidatorFactory	Genesis-pinned rules on what values may be written
Domain API	DomainApiProvider	Publish your own query endpoints through the node
Health / metrics	PluginHealthProvider · PluginMetricsProvider	Feed the node's health endpoint and Prometheus metrics

How a plugin is delivered

Implement the SPI, declare it in `META-INF/services`, add a bundle manifest, drop the jar in `plugins/`. The manifest declares `yanoApi {min, max, minLevel}`, and the catalog verifies digests and compatibility offline before anything loads. Config selects it by `type`.

Consensus-critical config must be identical everywhere

State machine, observers, sequencer mode, stability depth and commitment profile are part of the agreement. If one member is configured differently it will compute a different block and simply stop agreeing. The framework fails closed — it never guesses.

The three rules that make it trustworthy

1 · Determinism — the contract you must not break

`apply()` is called once per finalized block, in height order, **on every member**, and must produce identical writes everywhere. A nondeterministic machine does not corrupt the chain — it *stalls* it, because followers reject the state root.

Forbidden inside `apply()`

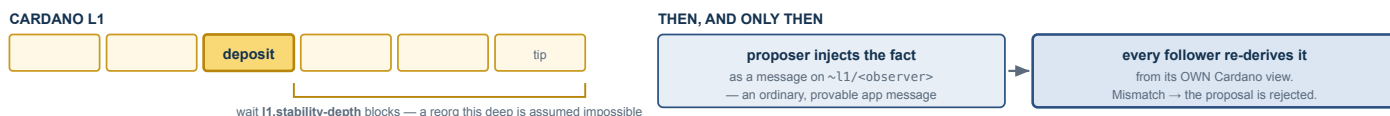
- Wall-clock time — use `block.timestamp()`
- Randomness, network or file I/O
- Environment / system-property reads
- `HashMap` / `HashSet` iteration — order is unspecified
- Locale- or charset-dependent serialization

How you check it

Run the `StateMachineConformance` harness before deploying. It is the cheapest bug you will ever prevent: a determinism defect found in production looks like a chain that mysteriously stopped producing blocks.

2 · Rollback — Cardano rolls back, the app chain does not

This asymmetry drives the whole L1-facing design. There is no un-apply path below finality, so an L1 fact must be **irreversible before it is allowed in**. The framework does this with a waiting period, not with compensation.



So a dishonest proposer cannot invent an L1 fact, cannot rush one in early, and cannot smuggle one that is not yet deep enough.
It supplies the data; it never gets to be believed. Everything the proposer asserts about Cardano is checked independently by everyone else.

3 · Proof — why an outsider should believe any of this

Anyone can verify a value from the app chain **without trusting whoever runs it**, and without the chain even being online, in two steps:

Step 1 — inclusion proof

A proof that `key → value` is present in the state trie under a given **state root**. Produced by the authenticated map (MPF or classic JMT).

Step 2 — the anchor

That state root appears in a **confirmed Cardano transaction**. Fetch it from any Cardano explorer or node; nothing about Yano needs to be trusted.

Where to start

If you want to...	Do this
See one running	<code>./showcase.sh</code> — a multi-chain cluster with one command
Run a chain with no code	Pick a shipped state machine (<code>ordered-log</code> , <code>kv-registry</code> , <code>eutxo-ledger</code>) and configure it
Write your own machine	Implement <code>AppStateMachine</code> , start from <code>scaffolds/plugin-template</code> , verify with <code>StateMachineConformance</code>
React to Cardano events	Configure a built-in observer, or implement <code>L1ObserverProvider</code> . Requires <code>l1.stability-depth > 0</code>
Read the detail	<code>docs/APP_CHAIN_USER_GUIDE.md</code> · <code>docs/APP_CHAIN_CONSENSUS_GUIDE.md</code> · <code>adr/app-layer/</code>

Three things newcomers get wrong

- **"The app chain follows Cardano's forks."** It does not. It is append-only below finality; L1 facts wait out the stability depth before they are allowed in.
- **"Only the proposer runs the state machine."** Every member runs it, twice — to verify a proposal and to commit the block. That is the whole security model.
- **"Config differences are harmless."** Consensus-critical config is part of the agreement. One member with a different state machine or observer set will simply stop agreeing.